EXPERIMENT - 7

Integrating a Zoho Creator Application with another Zoho Service using Connections and APIs

Insurance Claim Request & Processing Management

Application Name	Insurance Claim Request & Processing Management
Form Name	Claim_Request_Form
Experiment Focus	Zoho Mail connection and REST API
Student Name	HARIHARAN N
Student Register No.	241401024
Experiment	7
Platform	Zoho Creator
Course	Low-Code Application Development using Zoho Creator

Lab Record
An authorized Zoho Mail connection sends claim acknowledgements and status notifications from Creator through a reusable Deluge function. API responses and delivered messages provide end-to-end evidence.

Department of Computer Science and Business Systems

Rajalakshmi Engineering College

1. Objective

To integrate the insurance application with Zoho Mail using a Connection and an authenticated API request, then verify automatic acknowledgement and status emails.

2. Integration Requirement

Claimants need a reference number after submission and a notification when processing status changes. The mail service should be called from application workflows without embedding an OAuth token or password in Deluge code.

3. Connection Configuration

Setting	Implemented value
Service	Zoho Mail
Connection display name	Insurance Claims Mail
Connection link name	insurance_claims_mail
Permission	Zoho Mail Send Email
Authorization	Administrator-authorized OAuth connection
Application access	Insurance Claim Request & Processing Management enabled; unrelated app disabled
Sender mailbox	hariharan1807@zohomail.in
Reusable function	map Send_Claim_Mail_API(map request_details)

4. API Contract

Item	Value
Method	POST
Endpoint	https://mail.zoho.in/api/accounts/{account_id}/messages
Request header	Content-Type: application/json
Request body	fromAddress, toAddress, subject, content, mailFormat = plaintext
Response check	status.code == 200
Connection	insurance_claims_mail

5. Integration Flow and Procedure

1. Create the Zoho Mail connection with Send Email permission.

2. Complete OAuth authorization and mailbox setup under the owner account.

3. Enable the connection for the insurance application.

4. Create Send_Claim_Mail_API to map request parameters, guard missing values and invoke the Mail endpoint.

5. Call the function from WF_New_Claim_Submitted and WF_Claim_Status_Notification.

6. Inspect the function execution result and workflow logs. Verify messages in Zoho Mail and the Delivered status.

6. Error Handling and Delivery Semantics

The function skips blank and example.com recipients, returning a clear result. It rejects missing account/sender configuration before making a request. API errors are caught and logged without removing a saved claim. A successful API response indicates API acceptance; the separate Delivered status confirms mail-service delivery.

Notification	Subject / content
Acknowledgement	Claim Received - Ref {Claim_ID}; reference, Submitted status and sign-in guidance.
Status change	Claim Status Update - Ref {Claim_ID}; reference and current status.

Only references and status are included in the plain-text message. Detailed claim information stays in the application. No OAuth access token or password is reproduced in this report.

7. Reusable Deluge Function - Part 1

Actual saved source; line wrapping is for print.

```
map Send_Claim_Mail_API(map request_details)
{
  result = Map();
  account_id = ifnull(request_details.get("account_id"),"").toString().trim();
  from_address = ifnull(request_details.get("from_address"),"").toString().trim();
  to_address = ifnull(request_details.get("to_address"),"").toString().trim();
  if(to_address == "" || to_address.toLowerCase().endsWith("@example.com"))
  {
    result.put("success",false);
    result.put("status","Skipped: no live recipient or demo address.");
    return result;
  }
  if(account_id == "" || from_address == "")
  {
    result.put("success",false);
    result.put("status","Mailbox configuration required: account_id and from_address.");
    return result;
  }
  payload = Map();
  payload.put("fromAddress",from_address);
  payload.put("toAddress",to_address);
  payload.put("subject",ifnull(request_details.get("subject"),"Insurance Claim Notification"));
  payload.put("content",ifnull(request_details.get("content"),"Please sign in to view your claim
     status."));
  payload.put("mailFormat","plaintext");
  try
  {
    api_response = invokeurl
```

7. Reusable Deluge Function - Part 2

```
    [
      url :"https://mail.zoho.in/api/accounts/" + account_id + "/messages"
      type :POST
      parameters:payload.toString()
      headers:{"Content-Type":"application/json"}
      connection:"insurance_claims_mail"
    ];
    api_status = api_response.get("status");
    result.put("success",api_status.get("code") == 200);
    result.put("status",api_status);
  }
  catch (error)
  {
    result.put("success",false);
    result.put("status","Mail API call failed. Check mailbox, connection authorization and
    quota.");
    info "Mail API delivery failed; claim data remains saved.";
  }
  return result;
}
```

8. Test Results

Test	Observed result
Blank/example.com recipient	Skipped with explanatory result.
Missing mailbox configuration	Configuration-required result.
Direct live API test	Executed successfully; code 200.
Automatic acknowledgement / status	Workflow logs returned success and code 200.
Claim #10 status update	Inbox message contains Under Review; Delivered.
Claim #11 acknowledgement	Sent folder shows Delivered to the supplied policyholder email.

9. Outcome

The integration uses Zoho Mail, so no substitute Zoho service was required. Claims staff and policyholders triggered the same shared integration during separate-role tests.

10. Implementation Evidence - Connection

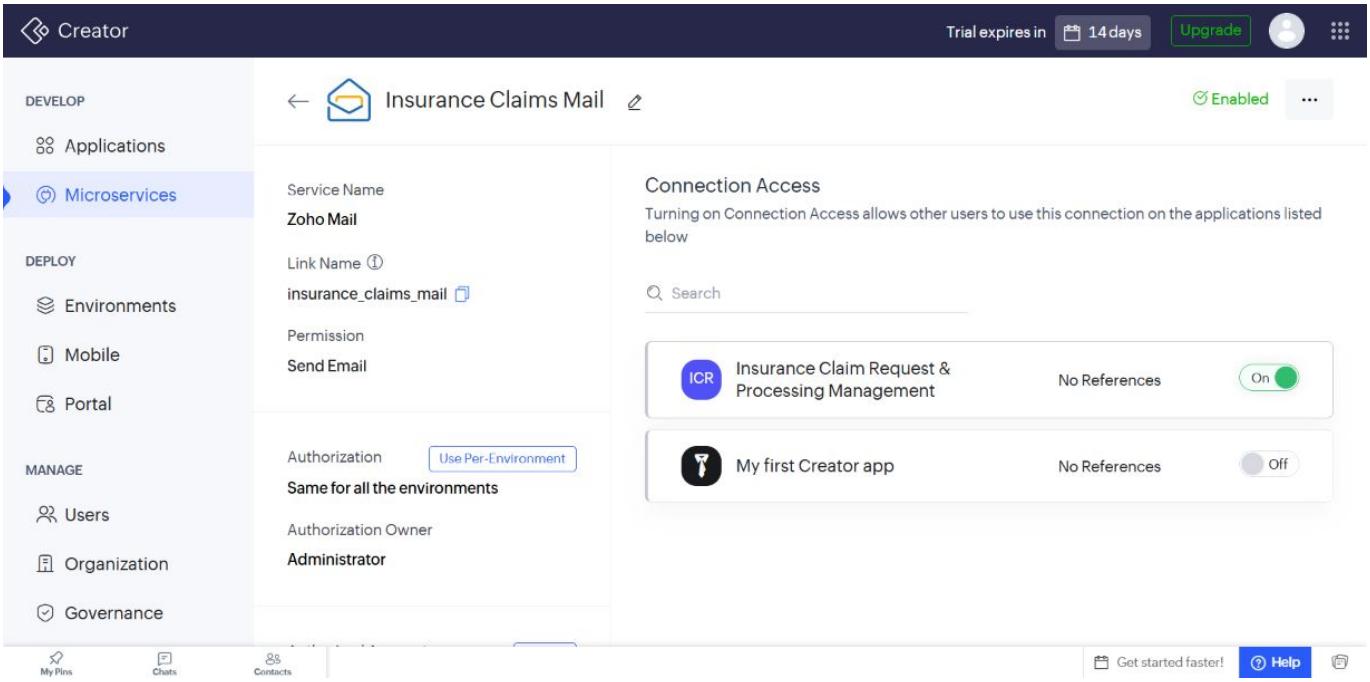


Figure 1. Insurance Claims Mail with Send Email permission and connection access enabled only for the insurance application.

10. Implementation Evidence - API Execution

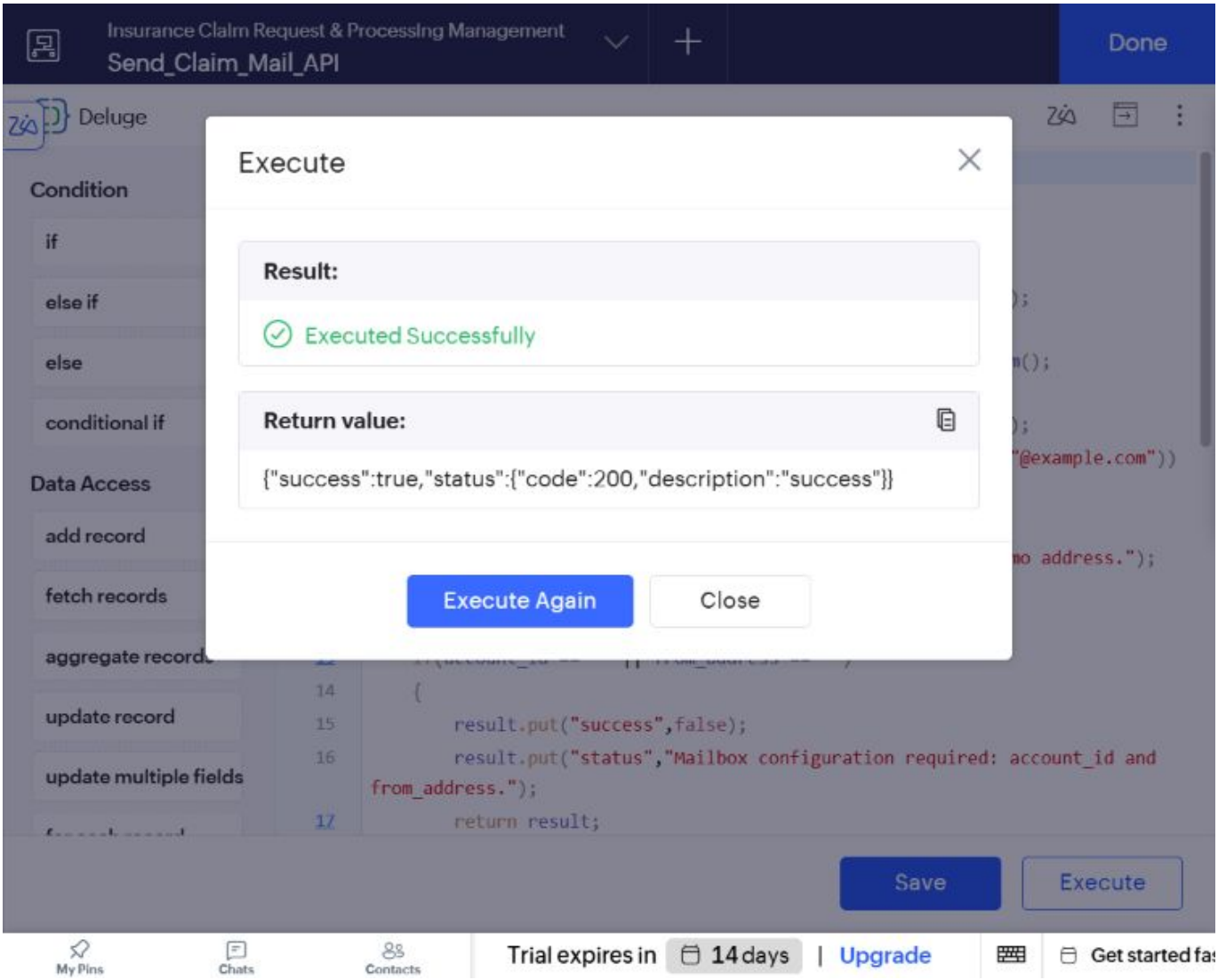


Figure 2. Reusable Mail API function executed successfully and returned code 200.

10. Implementation Evidence - Workflow Calls

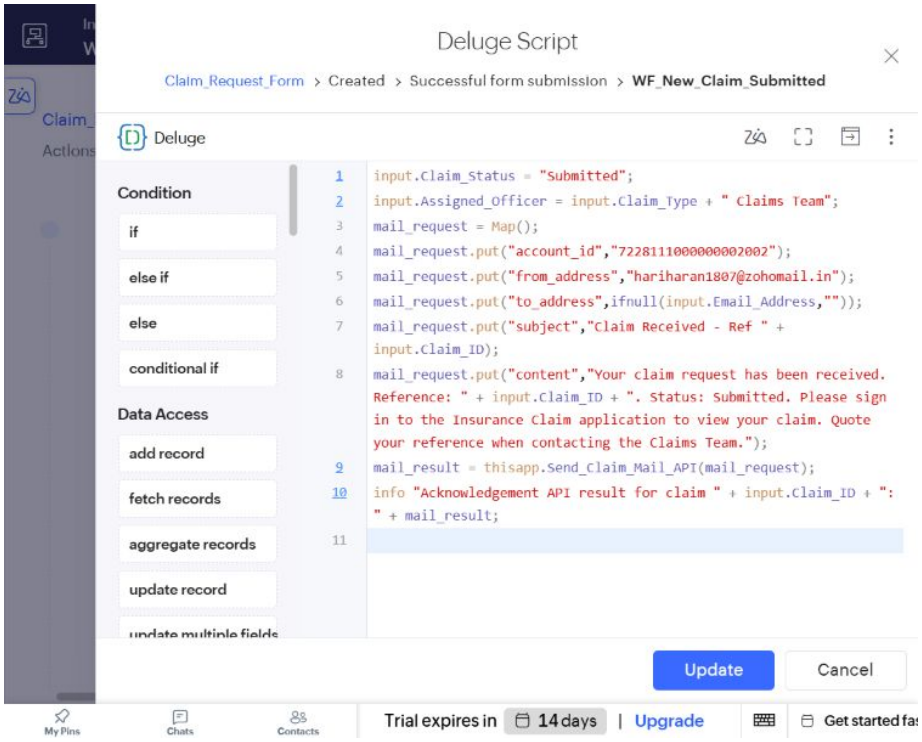


Figure 3. Acknowledgement workflow supplies reference, mailbox configuration and recipient.

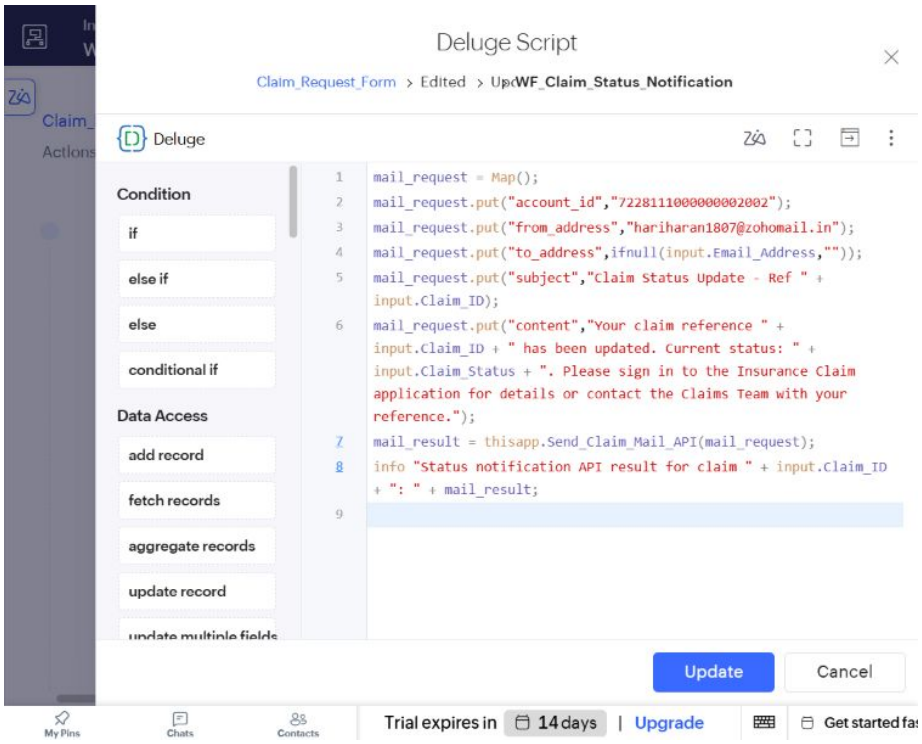


Figure 4. Status workflow calls the same function with the current claim status.

10. Implementation Evidence - API Logs

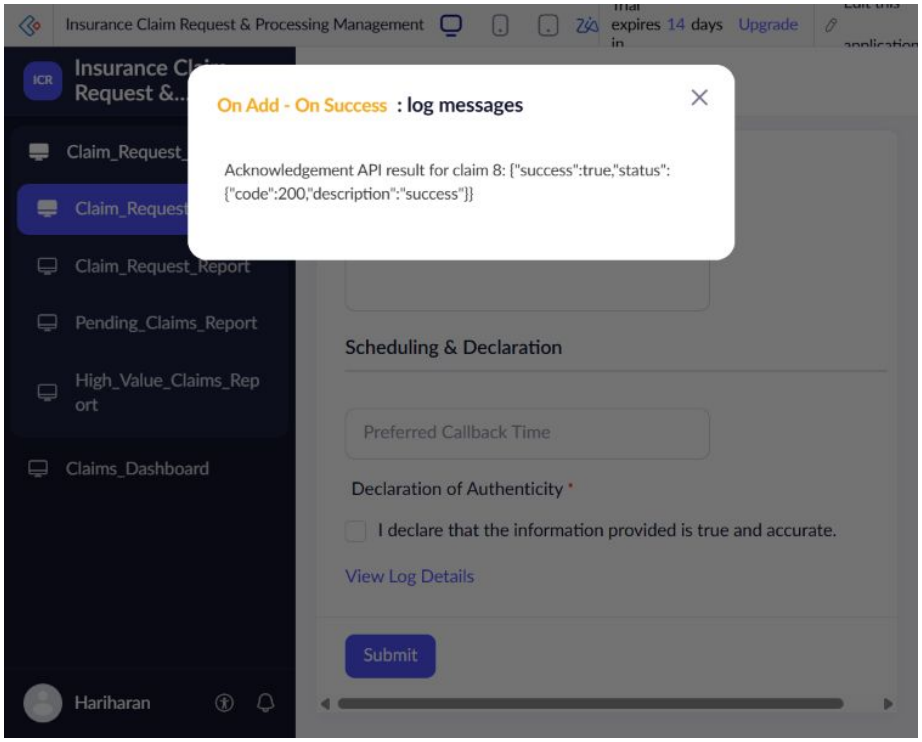


Figure 5. Automatic acknowledgement API success.

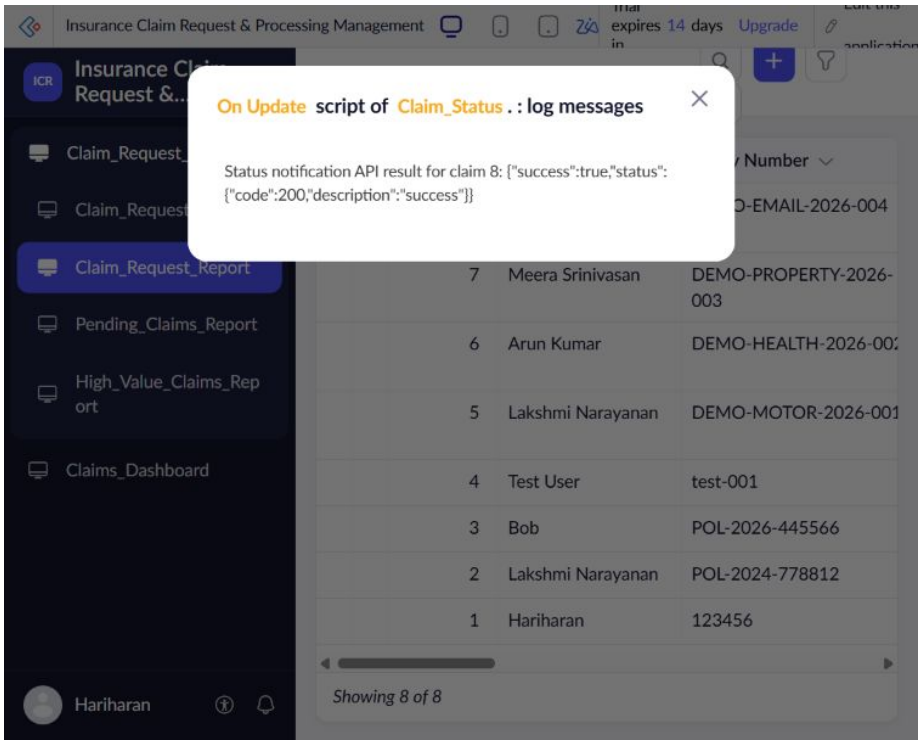


Figure 6. Automatic status notification API success.

10. Implementation Evidence - Delivered Mail

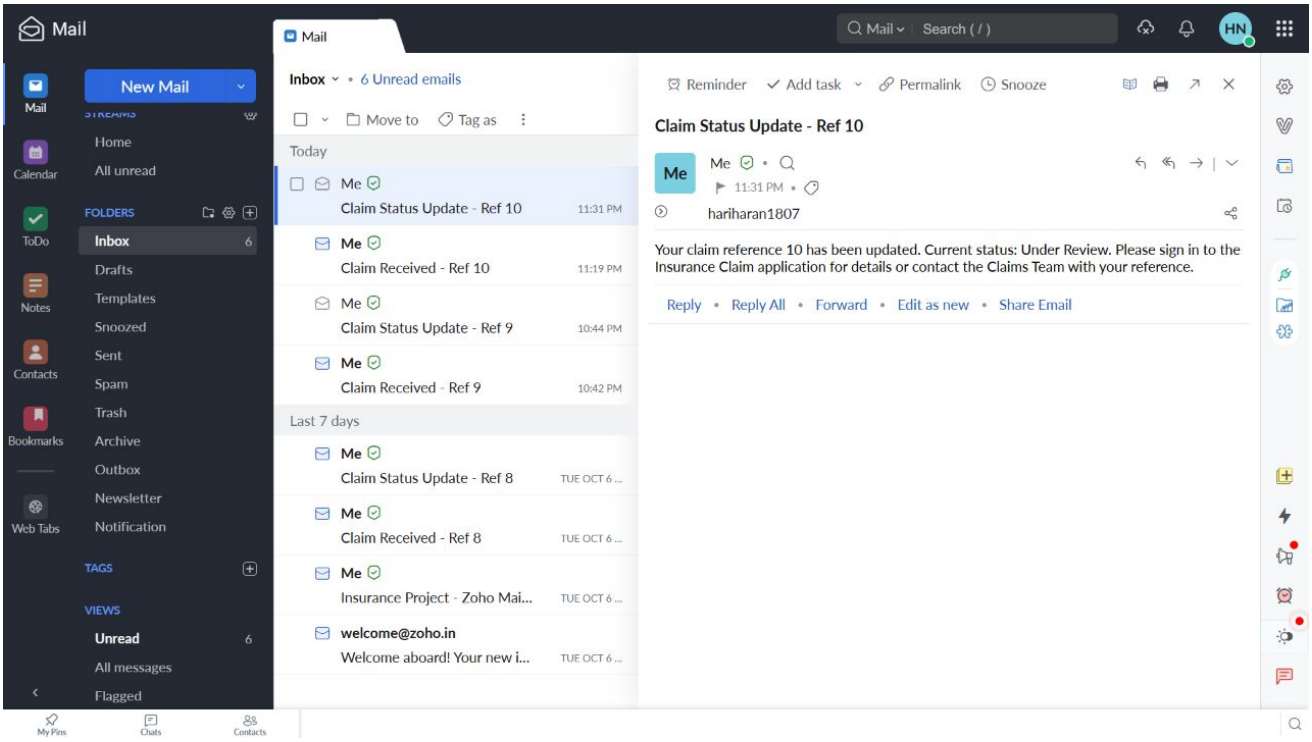


Figure 7. Received message for claim #10 shows Under Review.

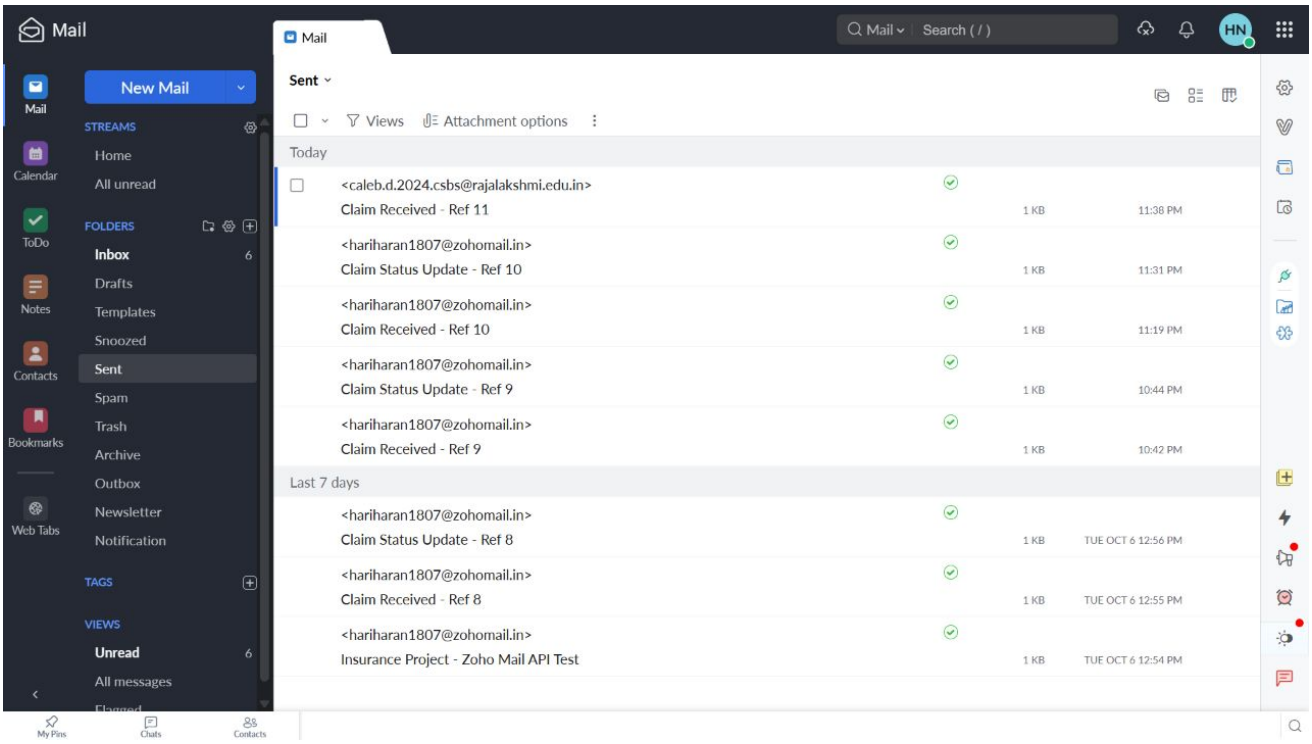


Figure 8. Sent folder confirms Delivered status for claim #11 acknowledgement and claim #10 status update.

11. Expected Learning Outcomes

1. Create and authorize a service connection with the required scope.

2. Use invokeurl with a Connection and a JSON request.

3. Reuse a Deluge function across multiple workflows.

4. Distinguish API success, handled errors and mail delivery evidence.

12. Result

The insurance application was integrated with Zoho Mail using insurance_claims_mail and the Mail messages API. Direct execution, workflow logs, message contents and delivery indicators verified the integration.

Conclusion

A single authenticated mail function supports both acknowledgement and status communication while keeping credentials out of the application script.

Observed scope: The original OAuth and mailbox setup were completed by the account owner. No external API key, password or token is printed in this report.

Department of Computer Science and Business Systems

Rajalakshmi Engineering College